Data analysis

Let's start by roughly analysing the tables and printing some characteristics.

In [72]:
```
import pandas as pd

prices1 = pd.read_csv('../data/prices1.csv')
prices2 = pd.read_csv('../data/prices2.csv')
weather = pd.read_csv('../data/weather.csv')

print('prices1 details')
print(f'Rows and Columns:\n{prices1.shape}\n')
print(f'Column Names:\n{prices1.columns.tolist()}\n')
print(f'Data types:\n{prices1.dtypes}\n')
print(f'Samples:\n{prices1.sample(5)}')
print('----------------------------------------------------------------------

print('prices2 details')
print(f'Rows and Columns:\n{prices2.shape}\n')
print(f'Column Names:\n{prices2.columns.tolist()}\n')
print(f'Data types:\n{prices2.dtypes}\n')
print(f'Samples:\n{prices2.sample(5)}')
print('----------------------------------------------------------------------

print('weather details')
print(f'Rows and Columns:\n{weather.shape}\n')
print(f'Column Names:\n{weather.columns.tolist()}\n')
print(f'Data types:\n{weather.dtypes}\n')
print(f'Samples:\n{weather.sample(5)}')
print('----------------------------------------------------------------------
```

```
prices1 details
Rows and Columns:
(16703, 11)

Column Names:
['delivery_date', 'id', 'business_date', 'time', 'time_interval', 'fixing_i_pric
e', 'fixing_i_volume', 'fixing_ii_price', 'fixing_ii_volume', 'continous_trading_
WAvg_price', 'continous_trading_volume']

Data types:
delivery_date                      str
id                               int64
business_date                      str
time                               str
time_interval                      str
fixing_i_price                 float64
fixing_i_volume                float64
fixing_ii_price                float64
fixing_ii_volume               float64
continous_trading_WAvg_price   float64
continous_trading_volume       float64
dtype: object

Samples:
                   delivery_date     id business_date      time time_interval  \
14255  2025-08-17 00:00:00+00:00  21645    2025-08-16  13:00:00         13-14
14044  2025-08-08 00:00:00+00:00  21435    2025-08-07  19:00:00         19-20
1900   2024-03-20 00:00:00+00:00  12605    2024-03-19  20:00:00         20-21
9350   2025-01-24 00:00:00+00:00  20026    2025-01-23  02:00:00           2-3
6788   2024-10-09 00:00:00+00:00  17465    2024-10-08  09:00:00          9-10

       fixing_i_price  fixing_i_volume  fixing_ii_price  fixing_ii_volume  \
14255          -20.00           3323.8            -3.55            2148.2
14044          591.00           3715.1           640.34            2578.4
1900           525.39           4769.9           567.94            1193.4
9350           397.66           2259.8           348.43             736.9
6788           429.99           2787.5           451.12            1622.2

       continous_trading_WAvg_price  continous_trading_volume
14255                        -26.61                     109.1
14044                           NaN                       0.0
1900                         535.00                     100.0
9350                         361.10                      11.3
6788                         434.99                      10.0
--------------------------------------------------------------------------------
--------------

prices2 details
Rows and Columns:
(5495, 19)

Column Names:
['delivery_date', 'id', 'business_date', 'time', 'time_interval', 'fixing_i_pric
e', 'fixing_i_volume', 'continous_trading_WAvg_price', 'continous_trading_volum
e', 'fixing_ii_price', 'fixing_ii_volume', 'fixing_ii_buy_volume', 'fixing_ii_sel
l_volume', 'total_listings_min_price', 'total_listings_max_price', 'total_listing
s_WAvg_price', 'total_listings_volume', 'total_listings_buy_volume', 'total_listi
ngs_sell_volume']

Data types:
```

```
delivery_date                       str
id                                int64
business_date                       str
time                                str
time_interval                       str
fixing_i_price                  float64
fixing_i_volume                 float64
continous_trading_WAvg_price    float64
continous_trading_volume        float64
fixing_ii_price                 float64
fixing_ii_volume                float64
fixing_ii_buy_volume            float64
fixing_ii_sell_volume           float64
total_listings_min_price        float64
total_listings_max_price        float64
total_listings_WAvg_price       float64
total_listings_volume           float64
total_listings_buy_volume       float64
total_listings_sell_volume      float64
dtype: object

Samples:
                  delivery_date    id business_date      time   time_interval  \
2057  2025-11-25 00:00:00+00:00  2058    2025-11-24  18:00:00  2025-11-25_H18
3373  2026-01-19 00:00:00+00:00  3398    2026-01-18  14:00:00  2026-01-19_H14
1683  2025-11-10 00:00:00+00:00  1684    2025-11-09  04:00:00    10-11-25_H04
4265  2026-02-25 00:00:00+00:00  4290    2026-02-24  18:00:00  2026-02-25_H18
4190  2026-02-22 00:00:00+00:00  4215    2026-02-21  15:00:00  2026-02-22_H15

      fixing_i_price  fixing_i_volume  continous_trading_WAvg_price  \
2057         1397.09           2938.9                       1414.45
3373          532.00           3201.7                           NaN
1683          419.55           2642.9                           NaN
4265          823.65           4544.9                        820.75
4190          416.00           4385.7                           NaN

      continous_trading_volume  fixing_ii_price  fixing_ii_volume  \
2057                      33.0          1606.41               NaN
3373                       NaN           527.00               NaN
1683                       NaN              NaN               NaN
4265                      34.5           760.12               NaN
4190                       NaN           399.67               NaN

      fixing_ii_buy_volume  fixing_ii_sell_volume  total_listings_min_price  \
2057                   NaN                    NaN                   1397.09
3373                   NaN                    NaN                    501.70
1683                   NaN                    NaN                    419.55
4265                   NaN                    NaN                    578.36
4190                   NaN                    NaN                    397.73

      total_listings_max_price  total_listings_WAvg_price  \
2057                   1705.48                    1456.71
3373                    563.92                     531.83
1683                    419.55                     419.55
4265                    848.88                     802.19
4190                    416.00                     410.28

      total_listings_volume  total_listings_buy_volume  \
2057               4150.775                   4150.775
3373               4006.050                   3810.050
```

```
1683               2642.900                   2642.900
4265               7019.925                   7019.925
4190               6732.875                   6732.875

      total_listings_sell_volume
2057                    3961.425
3373                    4006.050
1683                    2642.900
4265                    5640.125
4190                    4734.325
--------------------------------------------------------------------------------
--------------

weather details
Rows and Columns:
(355831, 8)

Column Names:
['czas', 'id', 'miasto', 'temperatura', 'wilgotnosc', 'opady', 'naslonecznienie',
'predkosc_wiatru']

Data types:
czas                   str
id                   int64
miasto                 str
temperatura        float64
wilgotnosc         float64
opady              float64
naslonecznienie    float64
predkosc_wiatru    float64
dtype: object

Samples:
                            czas      id     miasto  temperatura  wilgotnosc  \
157232  2024-12-30 04:00:00+00:00   43552     Kielce        -2.40        97.0
30227   2024-03-10 23:00:00+00:00  213180    Rzeszów         6.90        78.0
145590  2024-12-03 05:00:00+00:00   49971     Kraków        -5.00       100.0
277903  2025-10-05 23:00:00+00:00  263328     Kielce         7.44        53.0
342305  2026-03-11 19:00:00+00:00  342792  Białystok         9.34        74.0

        opady  naslonecznienie  predkosc_wiatru
157232    0.0              0.0             7.20
30227     0.3              0.0            21.60
145590    0.0              0.0             3.60
277903    0.0              0.0             2.32
342305    0.0              0.0             2.89
--------------------------------------------------------------------------------
--------------
```

Data pre-processing

One quick improvement is to remove unnecessary columns from prices*.csv, keeping only the temporal data and fixing_i_price, and to rename the weather columns in English for consistency with the others.

In [73]:
```
prices1.drop(columns=
             ['fixing_i_volume', 'fixing_ii_price', 'fixing_ii_volume',
              'continous_trading_WAvg_price', 'continous_trading_volume'],
             inplace=True)
prices2.drop(columns=['fixing_i_volume', 'fixing_ii_price', 'fixing_ii_volume',
                      'continous_trading_WAvg_price', 'continous_trading_volume'
                      'fixing_ii_buy_volume', 'fixing_ii_sell_volume', 'total_li
                        'total_listings_buy_volume', 'total_listings_max_price',
                          'total_listings_sell_volume', 'total_listings_volume']
                          inplace=True)
weather.rename(columns={'czas': 'time', 'id': 'id', 'miasto': 'city', 'temperatu
                        'wilgotnosc': 'humidity', 'opady': 'precipitation', 'nas
                        'predkosc_wiatru': 'wind_speed'}, inplace=True)
print(f'Columns left in prices1: {prices1.columns.tolist()}')
print(f'Columns left in prices2: {prices2.columns.tolist()}')
print(f'Renamed weather columns: {weather.columns.tolist()}')
```

```
Columns left in prices1: ['delivery_date', 'id', 'business_date', 'time', 'time_i
nterval', 'fixing_i_price']
Columns left in prices2: ['delivery_date', 'id', 'business_date', 'time', 'time_i
nterval', 'fixing_i_price']
Renamed weather columns: ['time', 'id', 'city', 'temperature', 'humidity', 'preci
pitation', 'sunny', 'wind_speed']
```

Now let's work a bit with the time fields.

For the price1 table we have the following fields, followed by the interpretation:

- **delivery_date** (*str*): when electricity is delivered
- **business_date** (*str*): when the market auction happened
- **time** (*str*): when electricity is auctioned/delivered
- **time_interval** (*str*): the time interval in which the electricity has been auctioned

The assumption is that people trade electricity on day x (business_date) and then the same electricity is delivered on day x+1 (delivery_date). For the time field we assume that electricity price is assigned in that hour and than delivered in the same hour.

For the price2 table we have the following fields:

- **delivery_date** (*str*)
- **business_date** (*str*)
- **time** (*str*)
- **time_interval** (*str*)

And for weather we have only: **time** (*str*)

In [74]:
```
print("Time values in prices1:")
print(f'{prices1[["delivery_date", "business_date", "time", "time_interval"]].he
print(f'{prices1["time"].unique()}\n')
print(f'{prices1["time_interval"].unique()}\n')
print("-----------------------------------------------------------------------

print("Time values in prices2:")
print(f'{prices2[["delivery_date", "business_date", "time", "time_interval"]].he
print(f'{prices2["time"].unique()}\n')
print(f'{prices2["time_interval"].unique()}\n')
```

```
Time values in prices1:
                delivery_date business_date      time time_interval
0   2024-01-01 00:00:00+00:00    2023-12-31  19:00:00         19-20
1   2024-01-01 00:00:00+00:00    2023-12-31  18:00:00         18-19
2   2024-01-01 00:00:00+00:00    2023-12-31  23:00:00         23-24
3   2024-01-01 00:00:00+00:00    2023-12-31  17:00:00         17-18
4   2024-01-01 00:00:00+00:00    2023-12-31  16:00:00         16-17
5   2024-01-01 00:00:00+00:00    2023-12-31  15:00:00         15-16
6   2024-01-01 00:00:00+00:00    2023-12-31  14:00:00         14-15
7   2024-01-01 00:00:00+00:00    2023-12-31  13:00:00         13-14
8   2024-01-01 00:00:00+00:00    2023-12-31  12:00:00         12-13
9   2024-01-01 00:00:00+00:00    2023-12-31  11:00:00         11-12
10  2024-01-01 00:00:00+00:00    2023-12-31  10:00:00         10-11
11  2024-01-01 00:00:00+00:00    2023-12-31  09:00:00          9-10
12  2024-01-01 00:00:00+00:00    2023-12-31  20:00:00         20-21
13  2024-01-01 00:00:00+00:00    2023-12-31  07:00:00           7-8
14  2024-01-01 00:00:00+00:00    2023-12-31  08:00:00           8-9
15  2024-01-01 00:00:00+00:00    2023-12-31  22:00:00         22-23
16  2024-01-01 00:00:00+00:00    2023-12-31  00:00:00           0-1
17  2024-01-01 00:00:00+00:00    2023-12-31  01:00:00           1-2
18  2024-01-01 00:00:00+00:00    2023-12-31  21:00:00         21-22
19  2024-01-01 00:00:00+00:00    2023-12-31  03:00:00           3-4

<StringArray>
['19:00:00', '18:00:00', '23:00:00', '17:00:00', '16:00:00', '15:00:00',
 '14:00:00', '13:00:00', '12:00:00', '11:00:00', '10:00:00', '09:00:00',
 '20:00:00', '07:00:00', '08:00:00', '22:00:00', '00:00:00', '01:00:00',
 '21:00:00', '03:00:00', '04:00:00', '05:00:00', '06:00:00', '02:00:00']
Length: 24, dtype: str

<StringArray>
['19-20', '18-19', '23-24', '17-18', '16-17', '15-16', '14-15', '13-14',
 '12-13', '11-12', '10-11',  '9-10', '20-21',   '7-8',   '8-9', '22-23',
   '0-1',   '1-2', '21-22',   '3-4',   '4-5',   '5-6',   '6-7',   '2-3',
 '23-00', '09-10', '08-09', '07-08', '06-07', '05-06', '04-05', '03-04',
 '02-03', '01-02', '00-01']
Length: 35, dtype: str

--------------------------------------------------------------------------------
--------------

Time values in prices2:
                delivery_date business_date      time time_interval
0   2025-09-01 00:00:00+00:00    2025-08-31  01:00:00  01-09-25_H01
1   2025-09-01 00:00:00+00:00    2025-08-31  02:00:00  01-09-25_H02
2   2025-09-01 00:00:00+00:00    2025-08-31  03:00:00  01-09-25_H03
3   2025-09-01 00:00:00+00:00    2025-08-31  04:00:00  01-09-25_H04
4   2025-09-01 00:00:00+00:00    2025-08-31  05:00:00  01-09-25_H05
5   2025-09-01 00:00:00+00:00    2025-08-31  06:00:00  01-09-25_H06
6   2025-09-01 00:00:00+00:00    2025-08-31  07:00:00  01-09-25_H07
7   2025-09-01 00:00:00+00:00    2025-08-31  08:00:00  01-09-25_H08
8   2025-09-01 00:00:00+00:00    2025-08-31  09:00:00  01-09-25_H09
9   2025-09-01 00:00:00+00:00    2025-08-31  10:00:00  01-09-25_H10
10  2025-09-01 00:00:00+00:00    2025-08-31  11:00:00  01-09-25_H11
11  2025-09-01 00:00:00+00:00    2025-08-31  12:00:00  01-09-25_H12
12  2025-09-01 00:00:00+00:00    2025-08-31  13:00:00  01-09-25_H13
13  2025-09-01 00:00:00+00:00    2025-08-31  14:00:00  01-09-25_H14
14  2025-09-01 00:00:00+00:00    2025-08-31  15:00:00  01-09-25_H15
15  2025-09-01 00:00:00+00:00    2025-08-31  16:00:00  01-09-25_H16
16  2025-09-01 00:00:00+00:00    2025-08-31  17:00:00  01-09-25_H17
```

```
17  2025-09-01 00:00:00+00:00    2025-08-31  18:00:00  01-09-25_H18
18  2025-09-01 00:00:00+00:00    2025-08-31  19:00:00  01-09-25_H19
19  2025-09-01 00:00:00+00:00    2025-08-31  20:00:00  01-09-25_H20

<StringArray>
['01:00:00', '02:00:00', '03:00:00', '04:00:00', '05:00:00', '06:00:00',
 '07:00:00', '08:00:00', '09:00:00', '10:00:00', '11:00:00', '12:00:00',
 '13:00:00', '14:00:00', '15:00:00', '16:00:00', '17:00:00', '18:00:00',
 '19:00:00', '20:00:00', '21:00:00', '22:00:00', '23:00:00', '00:00:00']
Length: 24, dtype: str

<StringArray>
[  '01-09-25_H01',   '01-09-25_H02',   '01-09-25_H03',   '01-09-25_H04',
   '01-09-25_H05',   '01-09-25_H06',   '01-09-25_H07',   '01-09-25_H08',
   '01-09-25_H09',   '01-09-25_H10',
 ...
 '2026-04-17_H15', '2026-04-17_H16', '2026-04-17_H17', '2026-04-17_H18',
 '2026-04-17_H19', '2026-04-17_H20', '2026-04-17_H21', '2026-04-17_H22',
 '2026-04-17_H23', '2026-04-17_H24']
Length: 5495, dtype: str
```

We can notice that time_interval is just a label for time, so we can combine it with delivery_date and merge that information into a new feature called **delivery_timestamp**.

We can also cast the weather time column to datetime.

In [75]:
```
# With dt.normalize() we set the time to 00:00:00, then we add the right time co
prices1["delivery_timestamp"] = pd.to_datetime(prices1["delivery_date"].astype(s
prices2["delivery_timestamp"] = pd.to_datetime(prices2["delivery_date"].astype(s
weather["time"] = pd.to_datetime(weather["time"])

print(prices1[["delivery_date", "business_date", "time", "time_interval", "deliv
print("-------------------------------------------------------------------------
print(prices2[["delivery_date", "business_date", "time", "time_interval", "deliv
print("-------------------------------------------------------------------------
print(weather[["time"]].head())
```

```
               delivery_date business_date      time time_interval  \
0  2024-01-01 00:00:00+00:00    2023-12-31  19:00:00         19-20
1  2024-01-01 00:00:00+00:00    2023-12-31  18:00:00         18-19
2  2024-01-01 00:00:00+00:00    2023-12-31  23:00:00         23-24
3  2024-01-01 00:00:00+00:00    2023-12-31  17:00:00         17-18
4  2024-01-01 00:00:00+00:00    2023-12-31  16:00:00         16-17

         delivery_timestamp
0 2024-01-01 19:00:00+00:00
1 2024-01-01 18:00:00+00:00
2 2024-01-01 23:00:00+00:00
3 2024-01-01 17:00:00+00:00
4 2024-01-01 16:00:00+00:00
--------------------------------------------------------------------------------
--------------

               delivery_date business_date      time time_interval  \
0  2025-09-01 00:00:00+00:00    2025-08-31  01:00:00  01-09-25_H01
1  2025-09-01 00:00:00+00:00    2025-08-31  02:00:00  01-09-25_H02
2  2025-09-01 00:00:00+00:00    2025-08-31  03:00:00  01-09-25_H03
3  2025-09-01 00:00:00+00:00    2025-08-31  04:00:00  01-09-25_H04
4  2025-09-01 00:00:00+00:00    2025-08-31  05:00:00  01-09-25_H05

         delivery_timestamp
0 2025-09-01 01:00:00+00:00
1 2025-09-01 02:00:00+00:00
2 2025-09-01 03:00:00+00:00
3 2025-09-01 04:00:00+00:00
4 2025-09-01 05:00:00+00:00
--------------------------------------------------------------------------------
--------------

                       time
0 2023-12-31 23:00:00+00:00
1 2024-01-01 00:00:00+00:00
2 2024-01-01 00:00:00+00:00
3 2024-01-01 00:00:00+00:00
4 2024-01-01 00:00:00+00:00
```

Let's check for duplicate rows and drop them if necessary:

In [76]:
```
print(f'Duplicates timestamps in prices1: {prices1["delivery_timestamp"].duplica
# Here we show an example of duplicated timestamp in prices1.
print(prices1[prices1["delivery_timestamp"].duplicated(keep=False) == True].sort
prices1.drop_duplicates(subset=["delivery_timestamp"], inplace=True)
print('-----------------------------------------------------------------------

print(f'Duplicates timestamps in prices2: {prices2["delivery_timestamp"].duplica
# Here we show an example of duplicated timestamp in prices2, none in this case
print(prices2[prices2["delivery_timestamp"].duplicated(keep=False) == True].sort
#prices2.drop_duplicates(subset=["delivery_timestamp"], inplace=True)
print('-----------------------------------------------------------------------

print(f'Duplicates timestamps in weather: {weather.duplicated(["time", "city"]).

prices1.drop(columns=["delivery_date", "time", "time_interval"], inplace=True)
prices2.drop(columns=["delivery_date", "time", "time_interval"], inplace=True)
```

```
Duplicates timestamps in prices1: 24
                   delivery_date     id business_date      time time_interval  \
16606  2025-11-24 00:00:00+00:00  24008    2025-11-23  00:00:00         00-01
16628  2025-11-24 00:00:00+00:00  23960    2025-11-23  00:00:00         00-01
16627  2025-11-24 00:00:00+00:00  23961    2025-11-23  01:00:00         01-02
16604  2025-11-24 00:00:00+00:00  24009    2025-11-23  01:00:00         01-02
16587  2025-11-24 00:00:00+00:00  24010    2025-11-23  02:00:00         02-03
16626  2025-11-24 00:00:00+00:00  23962    2025-11-23  02:00:00         02-03
16590  2025-11-24 00:00:00+00:00  24011    2025-11-23  03:00:00         03-04
16625  2025-11-24 00:00:00+00:00  23963    2025-11-23  03:00:00         03-04
16589  2025-11-24 00:00:00+00:00  24012    2025-11-23  04:00:00         04-05
16624  2025-11-24 00:00:00+00:00  23964    2025-11-23  04:00:00         04-05

       fixing_i_price        delivery_timestamp
16606          445.86 2025-11-24 00:00:00+00:00
16628          445.86 2025-11-24 00:00:00+00:00
16627          439.99 2025-11-24 01:00:00+00:00
16604          439.99 2025-11-24 01:00:00+00:00
16587          420.00 2025-11-24 02:00:00+00:00
16626          420.00 2025-11-24 02:00:00+00:00
16590          416.00 2025-11-24 03:00:00+00:00
16625          416.00 2025-11-24 03:00:00+00:00
16589          422.11 2025-11-24 04:00:00+00:00
16624          422.11 2025-11-24 04:00:00+00:00
--------------------------------------------------------------------------------
--------------

Duplicates timestamps in prices2: 0
Empty DataFrame
Columns: [delivery_date, id, business_date, time, time_interval, fixing_i_price,
delivery_timestamp]
Index: []
--------------------------------------------------------------------------------
--------------

Duplicates timestamps in weather: 0
```

Here we can see a summary of the remaining data:

In [77]:
```
print(prices1.head())
print("--------------------------------------------------------------------------
print(prices2.head())
print("--------------------------------------------------------------------------
print(weather.head())
```

```
      id business_date  fixing_i_price        delivery_timestamp
0  10708    2023-12-31          330.00 2024-01-01 19:00:00+00:00
1  10707    2023-12-31          336.11 2024-01-01 18:00:00+00:00
2  10712    2023-12-31          250.00 2024-01-01 23:00:00+00:00
3  10706    2023-12-31          340.00 2024-01-01 17:00:00+00:00
4  10705    2023-12-31          330.00 2024-01-01 16:00:00+00:00
--------------------------------------------------------------------------------
--------------

   id business_date  fixing_i_price        delivery_timestamp
0   1    2025-08-31          398.99 2025-09-01 01:00:00+00:00
1   2    2025-08-31          383.00 2025-09-01 02:00:00+00:00
2   3    2025-08-31          368.54 2025-09-01 03:00:00+00:00
3   4    2025-08-31          361.98 2025-09-01 04:00:00+00:00
4   5    2025-08-31          369.00 2025-09-01 05:00:00+00:00
--------------------------------------------------------------------------------
--------------

                       time      id       city  temperature  humidity  \
0 2023-12-31 23:00:00+00:00  277494     Puławy          4.4      94.0
1 2024-01-01 00:00:00+00:00  175728     Kielce          2.4      98.0
2 2024-01-01 00:00:00+00:00  226802   Warszawa          2.3      94.0
3 2024-01-01 00:00:00+00:00  155356  Białystok          0.3      93.0
4 2024-01-01 00:00:00+00:00  211501    Rzeszów          6.8      81.0

   precipitation  sunny  wind_speed
0            0.0    0.0         8.1
1            0.0    0.0         3.6
2            0.0    0.0        10.8
3            0.0    0.0         7.2
4            0.0    0.0        14.4
```

Data merging

We have reached the point where prices1 and prices2 have the same columns. Now we can concatenate them, reset the index, and then aggregate the values by delivery_timestamp to handle the overlapping time period between the tables.

After that we can join the weather data.

In [78]:
```
# Union all on prices1 and prices2
concat_prices = pd.concat([prices1, prices2], ignore_index=True)
print(f'Sample of rows to aggregate:\n{concat_prices[concat_prices.duplicated(su

concat_prices_agg = concat_prices.groupby("delivery_timestamp").agg({
    "business_date": "first",
    "fixing_i_price": "mean"
}).reset_index()

# After aggregation, we should not have any duplicates, but we check it just in
print(f'Number of duplicates: {concat_prices_agg.duplicated(subset=["delivery_ti

# Merge the aggregated prices with weather data on delivery_timestamp and time,
price_weather = pd.merge(left=concat_prices_agg,
                         right=weather,
```

```
                            left_on="delivery_timestamp",
                            right_on="time",
                            how="left").drop(columns=["time", "id"])
print(price_weather.head())
```

```
Sample of rows to aggregate:
          id business_date  fixing_i_price        delivery_timestamp
16702     24    2025-08-31          422.10 2025-09-01 00:00:00+00:00
14628  21992    2025-08-31          398.99 2025-09-01 00:00:00+00:00
16679      1    2025-08-31          398.99 2025-09-01 01:00:00+00:00
14629  21993    2025-08-31          383.00 2025-09-01 01:00:00+00:00
14630  21994    2025-08-31          368.54 2025-09-01 02:00:00+00:00
16680      2    2025-08-31          383.00 2025-09-01 02:00:00+00:00
16681      3    2025-08-31          368.54 2025-09-01 03:00:00+00:00
14631  21995    2025-08-31          361.98 2025-09-01 03:00:00+00:00
16682      4    2025-08-31          361.98 2025-09-01 04:00:00+00:00
14632  21996    2025-08-31          369.00 2025-09-01 04:00:00+00:00
Number of duplicates: 0
         delivery_timestamp business_date  fixing_i_price          city  \
0 2024-01-01 00:00:00+00:00    2023-12-31          236.11        Kielce
1 2024-01-01 00:00:00+00:00    2023-12-31          236.11      Warszawa
2 2024-01-01 00:00:00+00:00    2023-12-31          236.11     Białystok
3 2024-01-01 00:00:00+00:00    2023-12-31          236.11       Rzeszów
4 2024-01-01 00:00:00+00:00    2023-12-31          236.11  Zielona Góra

   temperature  humidity  precipitation  sunny  wind_speed
0          2.4      98.0            0.0    0.0         3.6
1          2.3      94.0            0.0    0.0        10.8
2          0.3      93.0            0.0    0.0         7.2
3          6.8      81.0            0.0    0.0        14.4
4          4.2      90.0            0.0    0.0        11.0
```

Feature extraction

Since training is out of scope for this task, a good approach is to create different versions of the final datasets using different setups. Here there are listed a series of ideas that need to be tested with a model to see which one performs the best:

- Keep a national average for each weather feature. This will lead to a simple dataset that will probably have lower performance.
- Keep data for each city and each type of weather feature. This will lead to a more complex dataset with potentially better performance.
- Use a hybrid approach combining the first two options. This would be the most complex solution, but its performance is still uncertain, ideally it should be better.

National approach

This dataset is the simplest one we can create. It is very human-readable, but it probably does not contain enough features. However, it is still worth trying.

In [ ]:

```
import numpy as np
# Now we create the national level features by aggregating across all cities for
national_features = price_weather.groupby("delivery_timestamp").agg({
    "business_date": "first",
    "fixing_i_price": "first",
    "temperature": ["mean", "min", "max", "std"],
    "wind_speed": ["mean", "min", "max", "std"],
    "humidity": ["mean", "min", "max", "std"],
    "precipitation": ["mean", "min", "max", "std"],
    "sunny": ["mean", "min", "max", "std"]
})

national_features.columns = [
    f"{col}_{stat}" for col, stat in national_features.columns
]

national_features = national_features.reset_index()

# Extract time-based features from delivery_timestamp
national_features["hour"] = national_features["delivery_timestamp"].dt.hour
national_features["dow"] = national_features["delivery_timestamp"].dt.dayofweek
national_features["month"] = national_features["delivery_timestamp"].dt.month

# Encode cyclical features for hour and day of week, this will help the model to
national_features["hour_sin"] = np.sin(2*np.pi*national_features["hour"]/24)
national_features["hour_cos"] = np.cos(2*np.pi*national_features["hour"]/24)

national_features["dow_sin"] = np.sin(2*np.pi*national_features["dow"]/7)
national_features["dow_cos"] = np.cos(2*np.pi*national_features["dow"]/7)

national_features["month_sin"] = np.sin(2*np.pi*national_features["month"]/12)
national_features["month_cos"] = np.cos(2*np.pi*national_features["month"]/12)

# Create lag features for the target variable (fixing_i_price_first)
national_features = national_features.sort_values("delivery_timestamp")

national_features["lag_24"] = national_features["fixing_i_price_first"].shift(24
national_features["lag_48"] = national_features["fixing_i_price_first"].shift(48
national_features["lag_72"] = national_features["fixing_i_price_first"].shift(72
national_features["lag_168"] = national_features["fixing_i_price_first"].shift(1
national_features["roll_mean_24"] = (
    national_features["fixing_i_price_first"]
    .shift(1)
    .rolling(24)
    .mean()
)
national_features["roll_mean_48"] = (
    national_features["fixing_i_price_first"]
    .shift(1)
    .rolling(48)
    .mean()
)
national_features["roll_std_24"] = (
    national_features["fixing_i_price_first"]
    .shift(1)
    .rolling(24)
    .std()
)
national_features["roll_std_48"] = (
```

```
    national_features["fixing_i_price_first"]
    .shift(1)
    .rolling(48)
    .std()
)

national_features.to_csv("../data/output/national_features.csv", index=False)
```

In [85]:
```
summary = pd.DataFrame({
    "null_count": national_features.isnull().sum(),
    "null_pct": (national_features.isnull().mean()*100)
})

summary = summary[summary["null_count"] > 0].sort_values("null_count", ascending
print(summary.head(20))
```

```
                   null_count  null_pct
roll_mean_48              191  0.949776
roll_std_48               191  0.949776
lag_168                   179  0.890104
humidity_std              132  0.656390
temperature_std           132  0.656390
precipitation_std         132  0.656390
wind_speed_std            132  0.656390
sunny_std                 132  0.656390
roll_mean_24              119  0.591745
roll_std_24               119  0.591745
wind_speed_max             97  0.482347
temperature_min            97  0.482347
temperature_mean           97  0.482347
sunny_max                  97  0.482347
humidity_max               97  0.482347
humidity_mean              97  0.482347
humidity_min               97  0.482347
wind_speed_mean            97  0.482347
wind_speed_min             97  0.482347
temperature_max            97  0.482347
```

Cities approach

The idea is to keep the weather conditions of all cities to understand which city influences the price the most (using feature selection).

In order to do that, we will pivot the city rows and create a new column for each city and each weather feature. In this way, it will be possible to use them as features.

In [ ]:
```
# Pivot the data to have city-specific weather features in separate columns
city_features = price_weather.pivot_table(
    index=["delivery_timestamp", "business_date", "fixing_i_price"],
    columns="city",
    values=["temperature", "humidity", "precipitation", "sunny", "wind_speed"]
)

city_features.columns = [
    f"{metric}_{city}"
    for metric, city in city_features.columns
]
```

```
city_features = city_features.reset_index()

# Extract time-based features from delivery_timestamp
city_features["hour"] = city_features["delivery_timestamp"].dt.hour
city_features["dow"] = city_features["delivery_timestamp"].dt.dayofweek
city_features["month"] = city_features["delivery_timestamp"].dt.month

city_features["hour_sin"] = np.sin(2*np.pi*city_features["hour"]/24)
city_features["hour_cos"] = np.cos(2*np.pi*city_features["hour"]/24)

city_features["dow_sin"] = np.sin(2*np.pi*city_features["dow"]/7)
city_features["dow_cos"] = np.cos(2*np.pi*city_features["dow"]/7)

city_features["month_sin"] = np.sin(2*np.pi*city_features["month"]/12)
city_features["month_cos"] = np.cos(2*np.pi*city_features["month"]/12)

# Create lag features for the target variable (fixing_i_price)
city_features = city_features.sort_values("delivery_timestamp")

city_features["lag_24"] = city_features["fixing_i_price"].shift(24)
city_features["lag_48"] = city_features["fixing_i_price"].shift(48)
city_features["lag_72"] = city_features["fixing_i_price"].shift(72)
city_features["lag_168"] = city_features["fixing_i_price"].shift(168)
city_features["roll_mean_24"] = (
    city_features["fixing_i_price"]
    .shift(1)
    .rolling(24)
    .mean()
)
city_features["roll_mean_48"] = (
    city_features["fixing_i_price"]
    .shift(1)
    .rolling(48)
    .mean()
)
city_features["roll_std_24"] = (
    city_features["fixing_i_price"]
    .shift(1)
    .rolling(24)
    .std()
)
city_features["roll_std_48"] = (
    city_features["fixing_i_price"]
    .shift(1)
    .rolling(48)
    .std()
)

city_features.to_csv("../data/output/city_features.csv", index=False)
```

In [82]:

```
summary = pd.DataFrame({
    "null_count": city_features.isnull().sum(),
    "null_pct": (city_features.isnull().mean()*100)
})

summary = summary[summary["null_count"] > 0].sort_values("null_count", ascending
print(summary.head(20))
```

```
                              null_count   null_pct
humidity_Zielona Góra               3735  18.673133
precipitation_Zielona Góra          3735  18.673133
temperature_Zielona Góra            3735  18.673133
wind_speed_Zielona Góra             3735  18.673133
sunny_Zielona Góra                  3735  18.673133
lag_168                              168   0.839916
lag_72                                72   0.359964
roll_mean_48                          48   0.239976
lag_48                                48   0.239976
roll_std_48                           48   0.239976
precipitation_Wrocław                 43   0.214979
humidity_Gorzów Wielkopolski          43   0.214979
humidity_Opole                        43   0.214979
humidity_Kielce                       43   0.214979
humidity_Poznań                       43   0.214979
humidity_Toruń                        43   0.214979
humidity_Rzeszów                      43   0.214979
precipitation_Gdańsk                  43   0.214979
precipitation_Rzeszów                 43   0.214979
precipitation_Toruń                   43   0.214979
```

Hybrid approach

Moreover, to use more of a hybrid approach, we can calculate the national weather conditions and add them for each timestamp.

In this way, at the end, we will have a table with a column for each weather condition for each city, but also a column for each weather condition across the entire nation.

In [ ]:
```
# Aggregate weather features across cities for each timestamp
agg = price_weather.groupby("delivery_timestamp").agg({
    "temperature": ["mean", "min", "max", "std"],
    "wind_speed": ["mean", "min", "max", "std"],
    "humidity": ["mean", "min", "max", "std"],
    "precipitation": ["mean", "min", "max", "std"],
    "sunny": ["mean", "min", "max", "std"]
})

agg.columns = [
    f"{col}_{stat}" for col, stat in agg.columns
]

agg = agg.reset_index()

# Merge the aggregated weather features back to the city_features dataframe
city_hybrid_features = city_features.merge(agg, on="delivery_timestamp")

city_hybrid_features.to_csv("../data/output/city_hybrid_features.csv", index=Fal
```

In [87]:
```
summary = pd.DataFrame({
    "null_count": city_hybrid_features.isnull().sum(),
    "null_pct": (city_features.isnull().mean()*100)
})

summary = summary[summary["null_count"] > 0].sort_values("null_count", ascending
print(summary.head(20))
```

```
                            null_count   null_pct
humidity_Zielona Góra             3735  18.673133
sunny_Zielona Góra                3735  18.673133
precipitation_Zielona Góra        3735  18.673133
temperature_Zielona Góra          3735  18.673133
wind_speed_Zielona Góra           3735  18.673133
lag_168                            168   0.839916
lag_72                              72   0.359964
lag_48                              48   0.239976
roll_std_48                         48   0.239976
roll_mean_48                        48   0.239976
precipitation_Toruń                 43   0.214979
precipitation_Wrocław               43   0.214979
humidity_Opole                      43   0.214979
humidity_Poznań                     43   0.214979
humidity_Rzeszów                    43   0.214979
humidity_Wrocław                    43   0.214979
humidity_Łódź                       43   0.214979
precipitation_Poznań                43   0.214979
precipitation_Opole                 43   0.214979
humidity_Olsztyn                    43   0.214979
0
```

Conclusions

At the end of each table creation, we can see a summary of the null values present in each table.

The percentages are really low so there is nothing to be worried about, except for some weather features in 'Zielona Góra'. This city can be either removed from the dataset or we can use an approach more similar to the following one:

In [88]:
```
# remove startup rows
city_hybrid_features = city_hybrid_features.iloc[168:].copy()

# fill remaining numeric nulls
num_cols = city_hybrid_features.select_dtypes(include="number").columns
city_hybrid_features[num_cols] = city_hybrid_features[num_cols].fillna(city_hybr

summary = pd.DataFrame({
    "null_count": city_hybrid_features.isnull().sum(),
    "null_pct": (city_features.isnull().mean()*100)
})

summary = summary[summary["null_count"] > 0].sort_values("null_count", ascending
print(summary.head(20))
```

```
Empty DataFrame
Columns: [null_count, null_pct]
Index: []
```

Also, let's take into consideration that lag features will naturally contain null values since they rely on previous values. That's why we just decided to remove the first 168 rows to not have this kind of problem.

We have to take into account that this is not the best solution for handling null values. Other methods could also be applied depending on the requirements of the problem. For example, we could remove rows containing null values or use the mean instead of the median, and so on.